

AI Video Detection & Origin Tracking System

Implementation Documentation Package

SRS + Database Schema + API Contract + UI/UX Specification + AI Evaluation Plan + AI Coding Prompts

Recommended Stack: ASP.NET Core Web API, React.js, PostgreSQL, Python AI Service, FFmpeg, Object Storage, Background Workers

Prepared for: Kiramat Khan

Purpose: Start-ready documents for building the project using Cursor AI, Codex, Claude, and similar AI coding tools

Important product claim: The system should estimate AI/manipulation probability and attempt to find the earliest known public appearance. It must not claim guaranteed original source or 100 percent truth.

Table of Contents

- 1. Executive Implementation Summary
- 2. MVP Scope and Product Boundaries
- 3. Software Requirements Specification (SRS)
- 4. Database ERD and Full Schema Plan
- 5. API Contract and Backend Modules
- 6. UI/UX Wireframe Specification
- 7. AI Detection and Evaluation Plan
- 8. Origin Tracking and Matching Plan
- 9. Security, Privacy, Compliance, and Legal Wording
- 10. AI-Assisted Development Guide for Cursor/Codex/Claude
- 11. Sprint Backlog, QA Plan, and Acceptance Criteria
- Appendix A. Status Enums and Scoring Rules
- Appendix B. Report JSON Example
- Appendix C. Recommended Repository Structure

1. Executive Implementation Summary

This package converts the high-level project analysis into practical implementation documents that can be used immediately for planning, database design, API development, UI design, and AI-assisted coding. The product is a web-first system where users upload videos or submit supported URLs. The backend validates files, queues processing jobs, extracts frames and metadata, runs AI detection, performs internal/external similarity checks, and returns a structured result report.

Area	Decision
Initial Product	Web application MVP first. Android and iOS apps should be Phase 2 after the AI/report quality is validated.
Backend	ASP.NET Core Web API with clean architecture style, Entity Framework Core, FluentValidation, Serilog, JWT/refresh tokens.
Frontend	React.js with TypeScript. Use React Query/Axios for API calls and a component library such as Ant Design, MUI, or Tailwind-based components.
Database	PostgreSQL. Store structured data in normalized tables and raw metadata/model output in JSONB fields.
Storage	Object storage for videos, frames, thumbnails, and PDF reports. Do not store raw videos in PostgreSQL.
Processing	Background workers. Controllers should only validate/upload/create jobs; workers should process videos.
AI Service	Python FastAPI service or ONNX Runtime based service for model inference. Backend calls AI service through internal API.
Origin Claim	Use "earliest known public appearance" instead of "original first upload".

2. MVP Scope and Product Boundaries

2.1 MVP Goal

The MVP should prove that the product can reliably accept videos, process them asynchronously, generate a useful AI/manipulation probability report, and store analysis history. Origin tracking should be limited and clearly marked as best-effort.

2.2 MVP Included Features

- Email/password login, signup, password reset, JWT access token, refresh token, and role-based access.
- Video upload from device with file size, duration, MIME type, and extension validation.
- Background job pipeline with statuses: Uploaded, Queued, Processing, Completed, Failed, Retried.
- Frame extraction and thumbnail generation using FFmpeg.
- Technical metadata extraction using FFmpeg/MediaInfo/ExifTool.
- Basic AI detection score through a model service or external AI service wrapper.
- Evidence generation: suspicious frames, metadata warnings, processing warnings, and confidence explanation.
- Internal duplicate/similar video matching using perceptual hashes.
- Limited public-source matching where legal and technically supported.
- Result dashboard, history page, and downloadable PDF report.
- Admin dashboard for users, videos, processing jobs, failed jobs, logs, and model versions.
- Basic credits/pay-per-report foundation so monetization can start early.

2.3 MVP Out of Scope

- Guaranteed first upload discovery across the entire internet.
- Large-scale crawling of social platforms without official permissions/APIs.
- Court-ready forensic certification in MVP.
- Mobile apps in the first development milestone unless the web MVP is stable.
- 100 percent AI detection claims.
- Automatic legal conclusion such as declaring a video fake as a factual statement.

2.4 Product Wording Rules

Bad wording	Use this instead
This video is fake.	This video is likely AI-generated or manipulated based on detected signals.
We found the original source.	We found the earliest known public/indexed appearance available to this system.
100% accurate AI detector.	Probability-based AI/manipulation analysis with confidence and evidence.
No watermark means fake.	No provenance/watermark was found; this alone does not prove the video is fake.

3. Software Requirements Specification (SRS)

3.1 User Roles

Role	Description	Key Permissions
Guest	Visitor who has not logged in.	View landing page, pricing, terms, signup/login.
User	Normal customer who uploads and analyzes videos.	Upload video, view own jobs/results/history, download own reports, manage account.
Admin	System operator.	View users/jobs/videos/logs/model versions, retry failed jobs, manage credits/plans.
Reviewer - Phase 2	Human analyst who can review low-confidence reports.	View assigned reports, add manual notes, approve/reject report status.
Enterprise Admin - Phase 2	Organization-level manager.	Manage organization users, credits, API keys, retention settings.

3.2 Functional Requirements - Authentication

ID	Priority	Requirement	Acceptance Criteria
AUTH-001	Must	User can create account with name, email, and password.	Email must be unique; password rules enforced; user is created inactive/active based on product decision.
AUTH-002	Must	User can login with email and password.	Valid credentials return access token, refresh token, user profile, and role.
AUTH-003	Must	User can request password reset email.	A reset token is generated and expires after configured time.
AUTH-004	Must	Refresh token flow is supported.	Expired access token can be renewed with valid refresh token; revoked/expired refresh token fails.
AUTH-005	Must	Admin role is protected.	Non-admin users cannot access admin APIs or admin frontend routes.
AUTH-006	Should	OAuth login may be added later.	Google/Apple login is available only after base auth is stable.

3.3 Functional Requirements - Video Upload

ID	Priority	Requirement	Acceptance Criteria
VID-001	Must	User can upload supported video formats.	MP4, MOV, AVI, MKV, WebM accepted if MIME/file validation passes.
VID-002	Must	System validates file size and duration.	Rejected files show clear reason before processing job is created.
VID-003	Must	Uploaded file is stored in private object storage.	Database stores only path/URL, size, type, duration, owner, and status.
VID-004	Must	Upload creates an analysis job.	Job is created with status Queued and initial progress 0.
VID-005	Should	Duplicate file hash is checked.	Exact duplicates can reuse previous result or warn user depending on business rule.

VID-006	Should	System supports upload progress UI.	Frontend shows percent uploaded and then job processing status.
---------	--------	-------------------------------------	---

3.4 Functional Requirements - Processing Pipeline

ID	Priority	Requirement	Acceptance Criteria
JOB-001	Must	Processing is asynchronous.	API request returns quickly; video analysis happens in background worker.
JOB-002	Must	Worker updates status and progress.	User sees Queued, Processing, Completed, Failed and current step.
JOB-003	Must	Failed jobs are logged.	Failure stores error message, retry count, step name, and stack/trace reference.
JOB-004	Must	Admin can retry failed jobs.	Retry increments retry count and creates/continues a background job safely.
JOB-005	Should	Pipeline is idempotent.	Re-running a job does not duplicate frames/results incorrectly.

3.5 Functional Requirements - AI Analysis

ID	Priority	Requirement	Acceptance Criteria
AI-001	Must	System extracts keyframes for AI analysis.	Frames are extracted at configured intervals and/or scene boundaries.
AI-002	Must	System calls AI inference service.	AI service returns visual score, confidence, model version, and frame-level results.
AI-003	Must	System calculates final label.	Final label is Likely Real, Likely AI-Generated, Edited/Manipulated, Suspicious, or Inconclusive.
AI-004	Must	System stores evidence items.	Evidence includes suspicious frame references, metadata warnings, score impact, and severity.
AI-005	Should	System supports model versioning.	Every result stores model version used for reproducibility.
AI-006	Should	Low-confidence results recommend manual review.	Confidence level and explanation are shown to user.

3.6 Functional Requirements - Origin Matching

ID	Priority	Requirement	Acceptance Criteria
ORG-001	Must	Generate perceptual hashes for frames.	pHash/dHash/aHash are generated and stored for selected frames.
ORG-002	Must	Internal matching finds similar previously analyzed videos.	Matches are ranked by hash similarity, duration similarity, and metadata closeness.
ORG-003	Should	External lookup attempts supported sources.	System records source_search_attempts with success/failure and limitations.
ORG-004	Must	Result wording uses earliest known public appearance.	UI and report avoid guaranteed origin claims.
ORG-005	Should	Source timeline is sorted oldest to newest.	Each source shows platform, URL, upload datetime if available, similarity and confidence.

3.7 Functional Requirements - Reports and History

ID	Priority	Requirement	Acceptance Criteria
REP-001	Must	User can view completed analysis report.	Report includes summary, evidence, metadata, origin timeline, warnings, recommendations.
REP-002	Must	User can download PDF report.	PDF is generated server-side or

			from HTML template and stored in object storage.
REP-003	Must	User can view analysis history.	History is paginated, filtered by status/date/label, and only shows own videos.
REP-004	Should	Report can be regenerated after model update.	Admin-only or paid feature later; stores new model version.

3.8 Functional Requirements - Credits and Monetization Foundation

ID	Priority	Requirement	Acceptance Criteria
PAY-001	Should	System supports user credits.	Each completed analysis consumes configured credits unless free trial applies.
PAY-002	Should	Admin can manually add credits.	Manual payment verification can be used before gateway integration.
PAY-003	Should	Pricing/plan information is visible.	Landing/pricing page explains free trial, pay-per-report, and packages.
PAY-004	Could	Payment gateway integration.	Stripe/Paddle/local gateway can be added after MVP validation.

3.9 Non-Functional Requirements

Category	Requirement
Performance	Small videos should be queued within seconds; processing time depends on duration/AI workload. No long-running processing inside HTTP controllers.
Scalability	Queue and worker design must allow scaling workers independently from the API.
Reliability	Failed steps must log actionable errors and support safe retry.
Security	All uploaded files must be validated, private, signed-url protected, and deleted based on retention policy.
Accuracy	Result must be probability-based with confidence and evidence. Do not use absolute claims.
Privacy	User videos may contain personal data. Require consent, secure storage, limited retention, and privacy policy.
Maintainability	Use clean module boundaries, DTOs, validation layer, repositories/services, and automated tests.
Observability	Use structured logs, correlation IDs, job logs, and monitoring dashboards.

4. Database ERD and Full Schema Plan

The database should be designed around users, videos, background jobs, extracted frames, hashes, metadata, AI results, evidence, source matches, reports, credits, and audit logs. Full SQL is provided as a separate database_schema.sql file in the package.

4.1 Main Entity Relationships

- users 1--N videos
- videos 1--N analysis_jobs
- videos 1--N video_frames
- video_frames 1--N frame_hashes
- videos 1--1 metadata_results
- videos 1--N ai_results
- ai_results 1--N evidence_items
- videos 1--N source_matches
- videos 1--N source_search_attempts
- videos 1--N reports

- model_versions 1--N ai_results
- users 1--N credit_transactions
- users 1--N audit_logs

4.2 Table Summary

Table	Purpose	Important Columns
users	User/admin accounts	id, name, email, password_hash, role, is_active, created_at
refresh_tokens	Secure refresh token storage	id, user_id, token_hash, expires_at, revoked_at
videos	Uploaded video records	id, user_id, original_name, file_url, file_size, duration, status
analysis_jobs	Processing job lifecycle	id, video_id, status, progress, current_step, retry_count, error_message
video_frames	Extracted frames and thumbnails	id, video_id, frame_url, timestamp_seconds, frame_index, is_keyframe
frame_hashes	Perceptual hashes for matching	id, video_id, frame_id, phash, dhash, ahash, hash_version
metadata_results	Technical metadata and warnings	id, video_id, codec, fps, resolution, encoder, warnings_json, raw_json
model_versions	Model reproducibility	id, name, version, model_type, is_active, accuracy_notes
ai_results	AI/manipulation output	id, video_id, model_version_id, scores, final_label, confidence
evidence_items	Explainability evidence	id, video_id, ai_result_id, type, title, description, severity, frame_id
source_matches	Origin/matching candidates	id, video_id, platform, url, upload_datetime, similarity_score, rank
source_search_attempts	External lookup audit	id, video_id, platform, query_type, status, response_summary, error_message
reports	Generated HTML/PDF reports	id, video_id, pdf_url, html_url, created_at
credit_transactions	Credits/pay-per-report foundation	id, user_id, amount, transaction_type, reason, reference
audit_logs	Security/admin logs	id, user_id, action, entity_type, entity_id, ip_address, old_value, new_value

4.3 Recommended Indexes

- users.email unique index.
- videos.user_id + created_at for history pagination.
- videos.status for admin dashboard filtering.
- analysis_jobs.video_id and analysis_jobs.status for worker/admin queries.
- frame_hashes.phash/dhash/ahash for matching. For MVP, exact/prefix indexes are enough; advanced matching may need specialized hash search strategy.
- source_matches.video_id + rank for timeline display.
- audit_logs.user_id + created_at and audit_logs.entity_type + entity_id.

4.4 Important Database Rules

- Never store raw video binary in PostgreSQL.
- Store raw AI/model output and raw metadata in JSONB for traceability, but keep important fields in normal columns for search/filtering.
- Every AI result must store model_version_id.
- Every report should be traceable to video_id and model/result version.
- Soft delete videos when needed; physically delete files based on retention job.
- Use audit_logs for admin actions and security-relevant user actions.

5. API Contract and Backend Modules

A starter OpenAPI YAML file is provided separately in the package. This section explains the main API contract and payload rules.

5.1 API Response Standard

```
{
  "success": true,
  "message": "Operation completed",
  "data": {},
  "errors": []
}
```

5.2 Error Response Standard

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    { "field": "file", "message": "Unsupported video format" }
  ]
}
```

5.3 Authentication Endpoints

Method	Path	Auth	Purpose
POST	/api/auth/signup	No	Create user account.
POST	/api/auth/login	No	Login and return tokens.
POST	/api/auth/refresh-token	No	Issue new access token.
POST	/api/auth/forgot-password	No	Send password reset email.
POST	/api/auth/reset-password	No	Reset password with token.
POST	/api/auth/logout	User	Revoke refresh token.
GET	/api/auth/me	User	Return current user profile.

5.4 Video and Job Endpoints

Method	Path	Auth	Purpose
POST	/api/videos/upload	User	Upload video and create analysis job.
GET	/api/videos/history?page=&pageSize=	User	Paginated video history.
GET	/api/videos/{videoid}	User	Video detail and current status.
DELETE	/api/videos/{videoid}	User	Request deletion based on retention policy.
GET	/api/jobs/{videoid}/status	User	Get job status, progress, current step.
POST	/api/admin/jobs/{jobid}/retry	Admin	Retry failed job.

5.5 Analysis, Origin, and Report Endpoints

Method	Path	Auth	Purpose
GET	/api/analysis/{videoid}/result	User	Final label, scores, confidence, warnings.
GET	/api/analysis/{videoid}/frames	User	Extracted frames and frame-level scores.
GET	/api/analysis/{videoid}/metadata	User	Technical metadata and warnings.
GET	/api/analysis/{videoid}/evidence	User	Evidence items with severity and score impact.
GET	/api/origin/{videoid}/matches	User	Timeline/source match list.
GET	/api/origin/{videoid}/earliest-known-source	User	Top earliest known source candidate.
POST	/api/reports/{videoid}/generate	User	Generate or regenerate PDF report.
GET	/api/reports/{videoid}/download	User	Download signed report URL or stream.

5.6 Admin Endpoints

Method	Path	Purpose
GET	/api/admin/dashboard	Counts, processing stats, failures, usage.
GET	/api/admin/users	User list with filters.
GET	/api/admin/videos	All video records with filters.
GET	/api/admin/jobs	All jobs by status/date.
GET	/api/admin/failed-jobs	Failed jobs list.
GET	/api/admin/audit-logs	Audit logs list.
GET	/api/admin/model-versions	Model version list.
POST	/api/admin/model-versions	Add model version metadata.
POST	/api/admin/users/{userId}/credits	Add/remove credits manually.

5.7 Upload API Request Rules

- Use multipart/form-data for MVP upload.
- Fields: file, user notes optional, analysis mode optional (basic/detailed), consent checkbox required.
- Validate extension, MIME type, duration, size, and whether file can be opened by FFmpeg.
- Return videoId and jobId immediately after job creation.
- Frontend should poll job status every 2-5 seconds during processing; use SignalR/WebSocket later if desired.

6. UI/UX Wireframe Specification

This is a text-based UI/UX specification. It is enough to give Cursor/Codex/Claude to generate React components. Later, a designer can convert it into Figma wireframes.

6.1 Design Principles

- Trust-first design: show confidence, evidence, limitations, and clear disclaimers.
- Do not make the result page look like a court verdict. It is an analysis report.
- Show progress clearly because video processing can take time.
- Make error messages friendly and actionable.
- Keep upload page simple: drag/drop, selected file details, consent, start analysis.
- Admin dashboard should focus on failed jobs, cost, usage, and operational health.

6.2 Public Landing Page

Section	Components
Hero	Headline, subtitle, upload CTA, login/signup buttons, short disclaimer.
How It Works	Upload video -> AI and metadata analysis -> similarity/origin check -> report.
Use Cases	Journalists, creators, agencies, legal teams, brands, researchers.
Sample Report	Preview card with AI probability, confidence, evidence, origin timeline.
Pricing	Free trial, pay-per-report, credit packages, enterprise contact.
Footer	Terms, privacy, contact, product limitation statement.

6.3 User Dashboard

- Top cards: remaining credits, total analyses, completed reports, failed jobs.
- Primary CTA: Upload New Video.
- Recent analysis table: thumbnail, filename, status, final label, confidence, date, action buttons.
- Filters: status, label, date range.
- Empty state: "Upload your first video to generate an analysis report."

6.4 Upload Screen

- Drag-and-drop upload box with Browse button.

- Supported formats and size/duration limits shown before upload.
- Consent checkbox: "I confirm I have the right to upload this video for analysis."
- Mode selector: Basic Analysis / Detailed Report (if credit system is enabled).
- Upload progress bar while transferring file.
- After upload: route to Processing screen using videoId/jobId.

6.5 Processing Screen

UI Area	Behavior
Status card	Shows Uploaded, Queued, Processing, Completed, Failed.
Progress bar	Displays numeric progress and current step.
Current step text	Examples: validating file, extracting metadata, extracting frames, running AI model, matching sources, generating report.
Failure state	Shows friendly error and "Contact support" or retry option if allowed.
Completion state	Button: View Report.

6.6 Result Detail Screen

Section	Content
Summary	Final label, AI probability, confidence level, date, model version, report download button.
Evidence	Evidence cards with title, severity, score impact, and frame preview if applicable.
Frame Analysis	Grid/list of frames with timestamp, score, notes.
Metadata	Codec, resolution, FPS, duration, bitrate, encoder, creation time, warnings.
Origin Timeline	Matched sources sorted by earliest known public appearance, similarity, confidence.
Limitations	Private/deleted/non-indexed sources may not be found; result is probability-based.
Recommendation	Likely AI, likely real, manual review recommended, or inconclusive.

6.7 Admin Dashboard

- Operational stats: total jobs, queued, processing, completed, failed, average processing time.
- Cost/usage stats: videos processed today, storage used, estimated AI calls.
- Failed jobs table with retry button and error step.
- Users table with credits, status, role, actions.
- Model versions table and active model indicator.
- Audit log viewer.

7. AI Detection and Evaluation Plan

7.1 AI Pipeline

1. Input: uploaded video file from private storage.
2. Preprocess: generate low-resolution processing copy, extract metadata, extract keyframes, optional scene detection.
3. Visual analysis: score selected frames for AI/manipulation artifacts.
4. Temporal analysis: detect flicker, consistency issues, motion/object persistence problems where possible.
5. Metadata analysis: inspect missing metadata, encoder, re-encoding, compression, timestamps, container anomalies.
6. Provenance check: inspect C2PA/Content Credentials or known watermark/provenance signals if available.
7. Optional audio analysis: synthetic voice/lip-sync mismatch in Phase 2.
8. Final scoring: combine signals into label, AI probability, confidence, warnings, and evidence items.

7.2 Scoring Formula - Initial MVP

Do not hard-code these weights forever. Store them in configuration and track model/scoring version. Initial MVP scoring can use:

Signal	Weight	Notes
Visual frame score	40%	Average/top suspicious frame score. Strongest MVP signal.
Temporal consistency	20%	May be weak in MVP if no strong temporal model is available.
Metadata score	15%	Suspicious metadata is evidence, not proof.
Provenance/watermark	10%	Positive provenance can reduce risk; missing provenance should not prove AI.
Origin/matching signals	10%	Existing trustworthy source can change confidence but not necessarily AI score.
Audio optional	5%	Add in Phase 2 when audio model is available.

7.3 Final Label Thresholds - Initial

Condition	Label	Confidence Guidance
AI score ≥ 0.80 and evidence count high	Likely AI-Generated	High if model confidence and evidence agree.
AI score 0.60 - 0.79	Suspicious / Possibly AI	Medium unless evidence is strong.
AI score 0.40 - 0.59	Inconclusive	Recommend manual review.
AI score < 0.40 and no major manipulation signals	Likely Real	Confidence depends on metadata/provenance/source signals.
Major metadata/editing signs but weak AI signs	Edited/Manipulated	Do not label AI unless visual/temporal evidence supports it.

7.4 Dataset Plan

Dataset Bucket	Examples	Purpose
Real camera videos	Phone/camera videos, different lighting, resolutions, compression levels.	Measure false positive rate on real content.
AI-generated videos	Videos from public AI generation tools or licensed datasets.	Measure detection recall.
Edited real videos	Trimmed, color corrected, watermarked, compressed, filtered videos.	Avoid confusing normal editing with AI generation.
Deepfake/face swap videos	Face manipulation samples where legally permitted.	Detect face-level manipulation.
Social reuploads	Downloads from platforms with compression/cropping.	Test origin matching and compression robustness.
Adversarial cases	Screen recordings, cropped versions, muted videos, low resolution.	Understand failure modes.

7.5 Evaluation Metrics

- Accuracy: overall correct classification, but do not rely on it alone.
- Precision for AI label: among videos labelled AI, how many are actually AI.
- Recall for AI label: among actual AI videos, how many are detected.
- False positive rate: real videos incorrectly labelled AI. This must be carefully monitored.
- False negative rate: AI videos incorrectly labelled real.
- F1 score: balanced precision/recall metric.
- ROC-AUC/PR-AUC: useful for threshold selection.
- Calibration/Brier score: checks whether probability scores are meaningful.
- Processing time per minute of video and cost per analysis.

7.6 Model Release Checklist

- Create model_versions record before running production analyses.

- Run evaluation dataset and store metric report.
- Compare false positives against previous model.
- Run 20-50 manually reviewed sample reports.
- Update scoring thresholds only after testing.
- Add release notes for admin users.
- Never silently overwrite old results without storing model version history.

8. Origin Tracking and Matching Plan

8.1 Internal Matching

- Generate selected frame hashes using pHash/dHash/aHash.
- Compare with previous analyzed videos in database.
- Rank matches using multiple signals: hash distance, number of matching frames, duration closeness, metadata closeness, audio hash in Phase 2.
- Store match candidates in source_matches with platform = Internal.
- Show user whether video appears to match a previously analyzed upload. Respect privacy: do not reveal another user private data unless product policy allows it.

8.2 External Matching

- Generate thumbnails/keyframes and search supported services/APIs only where legal and technically allowed.
- Use YouTube/public web lookup as initial MVP source where feasible.
- Record every lookup attempt in source_search_attempts.
- Show failures transparently: source blocked, no API access, no match found, rate limit, or not indexed.
- Do not scrape private/restricted platforms in MVP.

8.3 Match Ranking Formula - Initial

$$\text{match_score} = (\text{hash_similarity} * 0.50) + (\text{duration_similarity} * 0.15) + (\text{metadata_similarity} * 0.10) + (\text{thumbnail_similarity} * 0.15) + (\text{source_credibility} * 0.10)$$

8.4 Origin Result Warnings

- Private, deleted, restricted, or non-indexed uploads may not be detected.
- A reupload can appear earlier than a current viral copy but still may not be the true original.
- Edited/cropped/compressed videos can reduce matching confidence.
- Upload timestamps depend on platform availability and timezone handling.

9. Security, Privacy, Compliance, and Legal Wording

9.1 Security Requirements

- Use HTTPS everywhere.
- Use JWT access tokens with refresh tokens stored as hashes.
- Use role-based authorization for admin APIs.
- Validate extension, MIME type, file header, file size, and duration.
- Store videos in private object storage outside public web root.
- Use signed URLs with short expiry for report/video/frame access.
- Run malware scan where practical before deeper processing.
- Rate-limit upload and analysis endpoints.

- Log admin actions and security-sensitive user actions.
- Delete files based on retention policy and user deletion request rules.

9.2 Privacy and Legal Requirements

- Before upload, require user confirmation that they have rights/permission to analyze the video.
- Privacy policy must explain what is uploaded, how long it is stored, who can access it, and how deletion works.
- Terms must say results are probabilistic and not legal proof.
- Reports should include model version, processing date, and limitation statement.
- Avoid presenting unverified public uploader information as criminal/fraud proof.
- For enterprise/forensic use, add audit trail, chain-of-custody, and digital signature in Phase 2/3.

9.3 Standard Disclaimer Text for UI/Report

This analysis is probability-based and generated from available technical, visual, metadata, and matching signals. It does not guarantee that a video is AI-generated or real. Origin tracking is limited to supported public/indexed sources and should be described as the earliest known public appearance found by this system, not as a guaranteed original upload.

10. AI-Assisted Development Guide for Cursor/Codex/Claude

Use AI coding tools in small controlled tasks. Do not ask the AI to build the whole product in one prompt. Give it this documentation, then ask it to implement one module at a time with tests.

10.1 Golden Rules for AI Coding Tools

- Always generate DTOs, validators, service interfaces, service implementations, controllers, and tests per feature.
- Never process video inside API controllers. Create a job and let worker process it.
- Never store video bytes in PostgreSQL.
- Every API must validate ownership: users can access only their own videos/results.
- Every admin endpoint must require Admin role.
- Every processing step must update progress and log job step.
- Every AI result must store model version.
- Every external source lookup must store source_search_attempts record.
- Use consistent ApiResponse<T> shape for all endpoints.
- Write tests for validation and authorization before adding complex features.

10.2 Recommended Repo Structure

```
src/
  backend/
    AiVideoDetection.Api/
    AiVideoDetection.Application/
    AiVideoDetection.Domain/
    AiVideoDetection.Infrastructure/
    AiVideoDetection.Worker/
  ai-service/
    app/
    models/
    tests/
  web/
    src/
      api/
      components/
      pages/
      features/
      routes/
      hooks/
  docs/
```

database/
deployment/

10.3 Cursor/Codex Prompt - Backend Foundation

Build the ASP.NET Core Web API foundation for AI Video Detection. Use clean architecture with Domain, Application, Infrastructure, Api projects. Add JWT auth with refresh tokens, PostgreSQL EF Core, FluentValidation, Serilog, Swagger, role-based authorization, and an ApiResponse<T> wrapper. Create entities for users and refresh_tokens based on the documentation. Include migrations, DTOs, validators, services, controllers, and basic unit tests. Do not implement video processing yet.

10.4 Cursor/Codex Prompt - Video Upload Module

Implement the Video Upload module. Add videos and analysis_jobs entities, DTOs, validators, upload endpoint POST /api/videos/upload using multipart/form-data, file validation for extension, MIME, size, and consent flag. Store file through IObjectStorageService abstraction. Create analysis job with status Queued. Return videoId and jobId. Do not process video inside controller. Include ownership checks, error responses, and tests.

10.5 Cursor/Codex Prompt - Worker + FFmpeg Pipeline

Implement background processing worker for analysis_jobs. Use Hangfire for MVP. Worker should load queued job, update status/progress/current_step, run FFmpeg metadata extraction, generate thumbnail, extract frames at configured interval, store frame records and object storage paths. Add job logs and retry handling. Processing must be idempotent. Include interfaces for IVideoProcessingService and IFrameExtractionService.

10.6 Cursor/Codex Prompt - AI Service Integration

Implement AI inference integration. Create Python FastAPI ai-service with /health and /analyze-frames endpoint that accepts frame URLs or uploaded frame references and returns per-frame ai_score, confidence, notes, and model_version. In .NET backend create IAIInferenceClient. Store model_versions, ai_results, and evidence_items. Add final scoring service with configurable weights and thresholds. Use mock model first so pipeline works end-to-end.

10.7 Cursor/Codex Prompt - React Upload and Results UI

Build React TypeScript frontend pages: Login, Signup, Dashboard, UploadVideo, ProcessingStatus, ResultDetail, History, AdminDashboard. Use Axios/React Query. Upload page uses drag-and-drop, consent checkbox, progress bar, and redirects to processing page. Processing page polls /api/jobs/{videoId}/status. Result page shows summary, scores, evidence, metadata, origin timeline, warnings, and PDF download button.

10.8 Cursor/Codex Prompt - Admin + Credits

Implement Admin module with dashboard stats, users list, videos list, failed jobs, retry failed job, model versions, audit logs, and manual credit adjustment. Add credit_transactions table and service. Before analysis starts, check whether user has free trial or enough credits. For MVP manual payments are verified by admin and credits are added manually.

11. Sprint Backlog, QA Plan, and Acceptance Criteria

11.1 First 4-Week Sprint Plan

Week	Goal	Deliverables
1	Foundation	Repos, backend projects, frontend project, PostgreSQL, auth, roles, Swagger, base layout.
2	Upload + Jobs	Video upload, validation, object storage abstraction, videos table, jobs table, Hangfire queue.
3	Processing	FFmpeg metadata extraction, thumbnail/frame extraction, job progress, admin failed jobs.
4	End-to-end MVP skeleton	Frontend upload/status/history, mock AI result, basic report page, PDF placeholder.

11.2 QA Checklist

- Signup/login/logout/refresh token work correctly.
- Non-admin cannot access admin APIs.
- User cannot access another user video/report.
- Invalid file extension is rejected.
- Invalid MIME type is rejected.
- Oversized video is rejected.
- Corrupt video fails gracefully with clear error and job log.
- Queued job progresses to Completed for valid video.
- Failed job can be retried by admin.
- Frames and metadata are created exactly once on retry/idempotent run.
- Result report displays scores and limitations.
- PDF report generates and downloads correctly.
- Credits are consumed only after successful/defined analysis step based on business rule.

11.3 MVP Acceptance Criteria

- User can create account and login.
- User can upload a valid video and see processing status.
- System validates file and creates a background job without blocking UI.
- System extracts metadata and keyframes.
- System stores AI result from mock or real AI service with model version.
- System stores evidence items and warnings.
- System performs internal matching using hashes or placeholder service in early MVP.
- User can view completed report and download PDF.
- Admin can view failed jobs and retry them.
- All report wording uses probability/confidence and earliest known public appearance language.

Appendix A. Status Enums and Scoring Rules

Enum	Values
UserRole	Guest, User, Admin, Reviewer, EnterpriseAdmin
VideoStatus	Uploaded, Queued, Processing, Completed, Failed, Deleted
JobStatus	Queued, Processing, Completed, Failed, Retrying, Cancelled
FinalLabel	LikelyReal, LikelyAiGenerated, EditedManipulated, Suspicious, Inconclusive
ConfidenceLevel	Low, Medium, High
EvidenceType	VisualArtifact, TemporalIssue, MetadataWarning, ProvenanceSignal, AudioIssue, OriginMatch, ProcessingWarning
Severity	Low, Medium, High, Critical
CreditTransactionType	Add, Consume, Refund, Adjustment

Appendix B. Report JSON Example

```
{
  "videoId": 101,
  "summary": {
    "finalLabel": "LikelyAiGenerated",
    "aiProbability": 0.82,
    "confidence": "Medium",
    "modelVersion": "visual-mvp-0.1"
  },
}
```

```
"evidence": [
  { "type": "VisualArtifact", "severity": "High", "title": "Unnatural facial texture",
    "timestamp": 12.5 },
  { "type": "MetadataWarning", "severity": "Medium", "title": "Creation time missing" }
],
"originTimeline": [
  { "platform": "YouTube", "url": "https://example.com", "uploadDateTime": "2026-05-01T10:00:00Z",
    "similarityScore": 0.91 }
],
"limitations": [
  "Private, deleted, restricted, or non-indexed uploads may not be detected.",
  "This is a probability-based analysis, not a legal conclusion."
]
}
```

Appendix C. Recommended Repository Structure

```
ai-video-detection/
  README.md
  docs/
    implementation-package.pdf
    openapi.yaml
  database/
    database_schema.sql
  src/
    backend/
      ai-service/
      web/
  tests/
  deployment/
    docker-compose.yml
  nginx/
  scripts/
```

Final Implementation Advice

Start by building a working end-to-end skeleton with mock AI results. After upload, processing, status tracking, result display, and PDF generation work correctly, replace mock AI with real model inference. This approach lets Cursor/Codex/Claude generate code module-by-module without getting lost in the full complexity of AI, storage, jobs, and UI at the same time.